EXPERIMENT 06

HILL CLIMBING

Artificial Intelligence / Python Practical

Experiment	06
Title	Implementation of Simple Hill Climbing in Python
Language	Python
Search Type	Local Search (Heuristic-Guided)
Primary Data Structure	None (single current-node tracking)

1. Aim

To write and execute a Python program that implements the Simple Hill Climbing algorithm to search for a locally optimal node using heuristic values.

2. Objective

The objective is to understand how a local search algorithm improves a solution step by step by always moving toward the best immediate neighbour, and why this approach can get stuck at a point that is only locally — not globally — optimal.

3. Theory

Hill Climbing is a local search algorithm. It evaluates only the current node and its immediate neighbours using a heuristic function h(n), and always moves to the single best neighbour.

Unlike search algorithms that maintain a frontier of many candidate nodes, Hill Climbing keeps no memory of earlier nodes and does not backtrack. Once it moves forward, it never reconsiders an earlier decision.

The search stops as soon as no neighbour improves on the current node's heuristic value. This stopping point is called a local optimum. It is not guaranteed to be the best solution in the entire search space — the global optimum.

4. Graph Used in the Practical

For this practical, the following example graph and heuristic values are used (lower h(n) is better):

Node	Neighbours	h(n)
A	B, C	5
B	D, E	3
C	F	4
D	—	6
E	—	1

F	—	2

Start node: A

Graph relationships: A → B, C; B → D, E; C → F.

5. Algorithm

- Set the current node to the start node.
- Get the neighbours of the current node.
- If there are no neighbours, stop.
- Find the neighbour with the best (lowest) heuristic value.
- If that neighbour's heuristic value is not better than the current node's, stop — a local optimum has been reached.
- Otherwise, move to that neighbour and repeat from step 2.

6. Python Program

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

heuristic = {
    'A': 5,
    'B': 3,
    'C': 4,
    'D': 6,
    'E': 1,
    'F': 2
}

def hill_climbing(graph, heuristic, start):
    current = start
    path = [current]

    while True:
        neighbours = graph[current]

        if not neighbours:
            break

        next_node = min(neighbours, key=lambda node: heuristic[node])
```

```
        if heuristic[next_node] >= heuristic[current]:
            break

        current = next_node
        path.append(current)

    return path

start = 'A'
result = hill_climbing(graph, heuristic, start)

print("Hill Climbing Path:")
print(" -> ".join(result))
print("Final Heuristic Value:", heuristic[result[-1]])
```

7. Expected Output

```
Hill Climbing Path:
A -> B -> E
Final Heuristic Value: 1
```

8. Step-by-Step Execution

Step	Current Node	Neighbours (h)	Best Neighbour	Decision
1	A (h=5)	B (3), C (4)	B (3)	3 < 5 → move to B
2	B (h=3)	D (6), E (1)	E (1)	1 < 3 → move to E
3	E (h=1)	—	—	No neighbours → stop

The algorithm starts at A and always steps toward the neighbour with the lowest heuristic value. It reaches E, whose heuristic value (1) is the lowest in the entire graph for this path, so the local optimum found here is also the best possible outcome for this example.

9. Time and Space Complexity

In the worst case, Hill Climbing examines every neighbour of every node it visits before stopping, giving a time complexity of O(V + E), where V is the number of vertices and E is the number of edges — similar to a single-path graph traversal, since no node is revisited. The space complexity is O(1) for the core search, since only the current node and its immediate neighbours are considered at any time (the path list kept here for reporting is O(V) in the worst case).

10. Advantages

- Simple to understand and implement.
- Uses very little memory, since it does not store the whole search tree.
- Can find a solution quickly when the search space is well-behaved.

11. Disadvantages

- Can get stuck at a local optimum that is not the best overall solution.
- Can get stuck on a plateau, a flat region where neighbouring heuristic values are equal.
- Can get stuck at a ridge, where the optimal direction needs more than one single step.
- Has no memory of the search, so it cannot backtrack from a wrong step.
- Not guaranteed to find the global optimum or even a goal node.

12. Applications

- Optimization problems with a well-defined, improvable objective
- Scheduling and resource allocation
- Neural network weight tuning (conceptual basis for gradient-based methods)
- Circuit design and layout problems

13. Hill Climbing vs Best First Search

Feature	Hill Climbing	Best First Search
Explores	Only current node's neighbours	All frontier nodes seen so far
Data structure	None (single current node)	Priority queue
Backtracking	No	Effectively yes, via the frontier
Memory usage	Very low	Higher — keeps whole frontier
Optimality	Not guaranteed, can get stuck locally	Not guaranteed, explores more broadly

14. Viva Questions and Answers

1. What is Hill Climbing?

It is a local search algorithm that always moves from the current node to the best neighbouring node, stopping when no neighbour is better.

2. What is a local optimum?

A point where no neighbouring node improves on the current node, but which may not be the best solution in the whole search space.

3. Does Hill Climbing backtrack?

No. It has no memory of previous nodes and never reconsiders an earlier decision.

4. What are the main problems with Hill Climbing?

Getting stuck at a local optimum, a plateau, or a ridge.

5. What is a plateau in Hill Climbing?

A flat area of the search space where neighbouring nodes have equal heuristic values, giving the algorithm no clear direction to move.

6. Is Hill Climbing guaranteed to find the best solution?

No. It stops at the first local optimum it reaches, which may not be the global optimum.

7. What is the space complexity of Hill Climbing, and why is it low?

O(1) for the core search, because it only needs to track the current node and its immediate neighbours.

8. What is the difference between Hill Climbing and Best First Search?

Hill Climbing only looks at the current node's neighbours and cannot backtrack; Best First Search keeps a priority queue of the entire frontier.

15. Conclusion

Simple Hill Climbing was implemented in Python using a heuristic function that always moves toward the best available neighbour. The experiment demonstrates both the strength of the approach — reaching a good solution quickly with minimal memory — and its key weakness, since the algorithm stops permanently as soon as it reaches any local optimum, without verifying whether a better solution exists elsewhere in the graph.

16. Practical Record Note

This document is a clean, portfolio-ready practical record template. If your college-issued practical sheet uses a different graph, program, or output, those source-specific details should replace the corresponding sections before submission.